

주인님, 관점을 3개로 나누면 이렇게 보면 됩니다.

[클라이언트]

사용자 브라우저 / 팝업

[나의 서비스]

내 Spring 서버

Controller / Service / Session / HTML

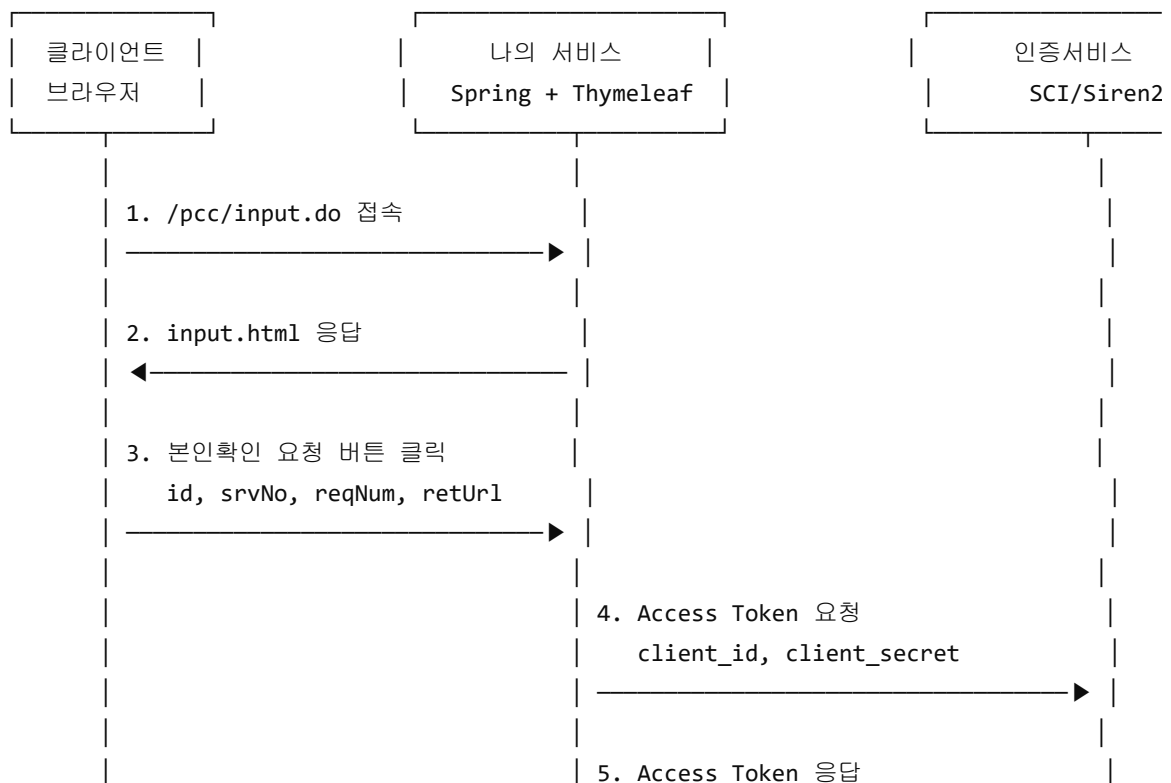
[인증서비스]

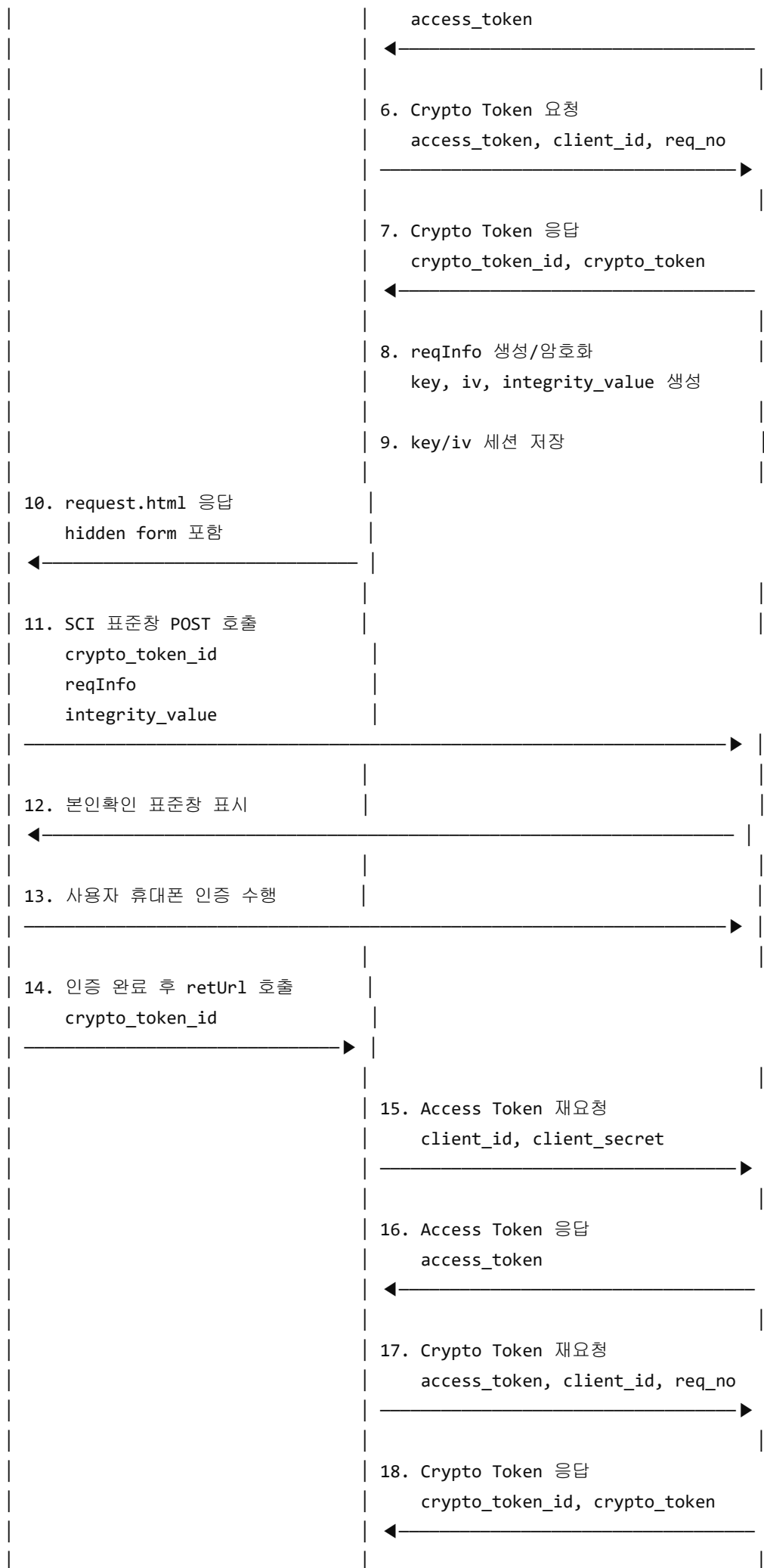
SCI / Siren24

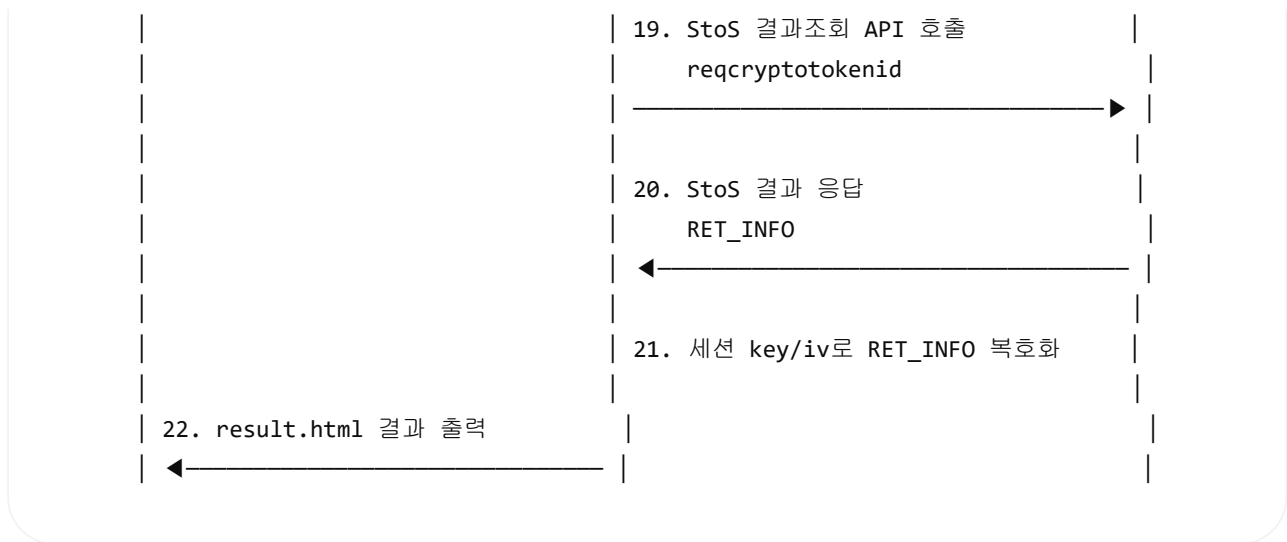
AccessToken API / CryptoToken API / 본인확인 표준창 / StoS API



전체 흐름 도식







단계별 핵심 데이터 역할

단계	데이터	위치	역할
3	id	클라이언트 → 나의 서비스	SCI 회원사 ID
3	srvNo	클라이언트 → 나의 서비스	SCI 서비스번호
3	reqNum	클라이언트 → 나의 서비스	이번 인증 요청 고유번호
3	retUrl	클라이언트 → 나의 서비스	인증 완료 후 돌아올 내 URL
4	client_id	나의 서비스 → SCI	API 호출 회원사 식별값
4	client_secret	나의 서비스 → SCI	Access Token 발급용 비밀 키
5	access_token	SCI → 나의 서비스	Crypto Token API 호출 권한
6	access_token	나의 서비스 → SCI	Bearer 인증값
6	req_no	나의 서비스 → SCI	Crypto Token 발급 기준 요청번호
7	crypto_token_id	SCI → 나의 서비스	암호화 토큰 식별자

단계	데이터	위치	역할
7	crypto_token	SCI → 나의 서비스	대칭키 생성 재료
8	key	나의 서비스 내부	AES 암호화/복호화 키
8	iv	나의 서비스 내부	AES CBC 초기화 벡터
8	reqInfo	나의 서비스 → SCI 표 준창	인증 요청정보 암호문
8	integrity_value	나의 서비스 → SCI 표 준창	요청 위변조 검증값
9	key , iv	Session	결과 복호화를 위해 임시 보관
11	reqInfo	클라이언트 → SCI 표 준창	암호화된 인증 요청정보
11	crypto_token_id	클라이언트 → SCI 표 준창	SCI가 요청 암호 해석할 때 참조
14	crypto_token_id	SCI 표준창 → 나의 서 비스	인증 완료된 거래 식별값
19	reqcryptotokenid	나의 서비스 → SCI StoS	결과 조회 대상 인증건
20	RET_INFO	SCI StoS → 나의 서비 스	최종 인증결과 암호문
21	decryptData	나의 서비스 내부	복호화된 최종 본인확인 결 과

요청 시작 구간

클라이언트

|

| 1. /pcc/input.do

▼

나의 서비스

|



```
| input.html 반환
▼
클라이언트
|
| id, srvNo, reqNum, returnUrl 입력 후 submit
▼
나의 서비스 /pcc/request.do
```

여기서 중요한 건 returnUrl 입니다.

```
returnUrl = https://도메인/context/pcc/result.do
```



returnUrl 은 인증 완료 후 SCI가 호출할 주소입니다.
/request.do 가 아니라 /result.do 입니다.

인증 요청 준비 구간

```
나의 서비스
|
| client_id + client_secret
▼
SCI Access Token API
|
| access_token 반환
▼
나의 서비스
|
| access_token + client_id + req_no
▼
SCI Crypto Token API
|
| crypto_token_id + crypto_token 반환
▼
나의 서비스
```



여기서 역할은 이렇습니다.

```
client_id
= 우리 서비스가 누구인지 식별

client_secret
= Access Token 발급용 비밀값

access_token
= Crypto Token API를 호출할 수 있는 권한값
```



crypto_token

= reqInfo 암호화 key/iv를 만들기 위한 재료

crypto_token_id

= SCI가 이 crypto_token을 식별하기 위한 ID

암호화 구간

나의 서비스 내부

req_date + req_no + crypto_token

↓

SHA-256

↓

Base64

↓

symmetricKey

↓

key = 앞 16자리

iv = 뒤 16자리

그리고 인증 요청 정보를 만듭니다.

```
{
  "id": "회원사ID",
  "srvNo": "서비스번호",
  "reqNum": "요청번호",
  "retUrl": "https://도메인/context/pcc/result.do",
  "certDate": "요청일시",
```

</> JSON

이걸 AES로 암호화하면:

reqInfo

그리고 위변조 검증값:

integrity_value

를 만듭니다.

표준창 호출 구간

클라이언트 브라우저



```
request.html
|
| hidden form 자동 submit
▼
SCI 표준창
```

전송되는 값:

```
<input type="hidden" name="crypto_token_id">
<input type="hidden" name="integrity_value">
```

HTML

역할:

```
crypto_token_id
= SCI가 어떤 crypto_token 기준으로 복호화할지 찾는 값

reqInfo
= 암호화된 인증 요청 정보

integrity_value
= reqInfo가 중간에 바뀌지 않았는지 확인하는 값

verSion
= API 방식 버전값
```



인증 완료 후 결과 수신 구간

SCI 표준창



```
|
| 인증 성공
|
| retUrl 호출
▼
나의 서비스 /pcc/result.do
```

SCI가 넘겨주는 핵심값:

crypto_token_id



이 값을 쉽게 말하면:

방금 인증 완료된 거래를 찾기 위한 식별값



입니다.

결과 조회 StoS 구간

나의 서비스



Access Token 재발급



SCI Access Token API



access_token



나의 서비스



Crypto Token 재발급



SCI Crypto Token API



crypto_token_id + crypto_token



나의 서비스



StoS 요청



SCI StoS API



RET_INFO



나의 서비스



StoS 요청에는 대략 이런 값이 들어갑니다.

```
{
  "dataHeader": {
    "CNTY_CD": "kr",
    "TRAN_ID": "비즈사이렌 로그인 아이디"
  },
  "dataBody": {
    "crypto_token_id": "StoS용 crypto_token_id",
```



```
"reqInfo": "암호화된 결과조회 요청정보",  
"integrity_value": "무결성값"
```

</> JSON



이때 reqInfo 내부에는:

```
{  
  "id": "비즈사이렌 로그인 아이디",
```

</> JSON



가 들어갑니다.

역할:

```
reqcryptotokenid  
= 어떤 인증건의 결과를 조회할지 지정하는 값
```



최종 복호화 구간

SCI StoS 응답



RET_INFO = 암호화된 최종 인증결과

나의 서비스는 세션에 저장해둔 값으로 복호화합니다.

```
Session  
├ PCC_KEY  
└ PCC_IV
```



복호화 후 최종 결과 예시는 이런 느낌입니다.

```
{  
  "name": "홍길동",  
  "birth": "19900101",  
  "sex": "1",  
  "ci": "개인식별연계정보",  
  "di": "중복가입확인정보",
```

</> JSON



가장 중요한 흐름만 압축

1. 사용자가 본인확인 시작
2. 나의 서비스가 Access Token 발급
3. 나의 서비스가 Crypto Token 발급
4. 나의 서비스가 reqInfo 암호화
5. 클라이언트가 SCI 표준창 호출
6. 사용자가 휴대폰 인증
7. SCI가 returnUrl=/result.do 호출
8. 나의 서비스가 StoS 결과조회
9. RET_INFO 수신
10. 세션 key/iv로 복호화
11. 최종 본인확인 결과 사용



한 줄 요약

Access Token

= API 호출 권한

Crypto Token

= reqInfo 암호화용 재료

reqInfo

= 인증 요청정보 암호문

crypto_token_id

= SCI가 암호화 토큰/인증거래를 찾는 식별값

returnUrl

= 인증 완료 후 돌아올 내 result.do

StoS

= 인증 완료 후 실제 결과를 서버끼리 조회하는 API

RET_INFO

= 최종 본인확인 결과 암호문



출처

응 잘 이해됐어.. 근데 왜 api 호출을 3번을하는거야 ?
각각 역할을 구분해 보안을 강화한 측면인가 ?

네 주인님. 정확히 그 관점으로 이해하면 됩니다.

결론부터 말하면:

AS-IS

= 암호화 모듈 신뢰



TO-BE

= 인증서버(API) 신뢰

로 바뀐 겁니다.

왜 굳이 3번 호출할까?

처음 보면 이런 생각이 들죠.

1. Access Token API

2. Crypto Token API

3. StoS 결과조회 API



아니 그냥 인증창 띄우고 결과주면 되는데
왜 3번이나 API를 호출하지?

사실은

각 API가 담당하는 보안 역할이 다르다



입니다.

1단계 Access Token API

목적

"너 누구냐?"



검증

예전 방식

내 서버



```
|
├ SCI 암호화 DLL
├ CLIENT_ID
└ CLIENT_SECRET
```

만 있으면

암호화 가능

인증창 호출 가능

문제

서버 해킹



↓

CLIENT_ID 탈취

↓

누구나 SCI 호출

가능

그래서

Access Token



개념 추가

비유

회사 출입문



사원증

↓

출입카드 발급

↓

건물 출입

처럼

CLIENT_ID
↓
Access Token
↓
Crypto Token API 사용



구조

2단계 Crypto Token API

목적

"이번 거래에만 사용할 암호화 재료 발급"



예전 방식

고정 암호화 키



사용

문제

1건 탈취
↓
과거 거래 복호화 가능
↓
미래 거래도 위험



그래서

매 거래마다

crypto_token
crypto_token_id



새로 발급

예시

사용자A 인증



crypto_token
= AAAA

사용자B 인증

crypto_token
= BBBB

사용자C 인증

crypto_token
= CCCC

전부 다름

따라서

A 거래 해킹



되어도

B,C 거래 영향 없음



3단계 StoS API

이게 제일 중요

예전 방식

SCI



브라우저



우리 서버



로 결과 전달

문제

브라우저는

사용자 영역



임

즉

개발자도구

프록시

해킹툴



가능

예시

SCI

↓

이름=홍길동

생년월일=19900101

CI=xxxx



브라우저 전달

↓

중간 변조 가능

그래서

TO-BE

SCI

↓

브라우저

↓

crypto_token_id만 전달

↓

우리 서버

↓

StoS API



↓
SCI

구조

즉

민감정보



는

서버 ↔ 서버



로만 이동

그래서 각각 역할

API	역할	보안 목적
Access Token	회원사 인증	무단 API 사용 방지
Crypto Token	거래별 암호화 재료	거래 단위 분리
StoS	결과 조회	브라우저 노출 차단

만약 Access Token 없으면?

CLIENT_ID 유출

↓

누구나 API 호출



가능

Crypto Token 없으면?

암호화키 고정

↓



한 번 유출되면
전체 거래 위험

StoS 없으면?

CI
DI
성명
생년월일
휴대폰



이

브라우저



를 거쳐야 함

현재 정부기관·금융권이 가장 싫어하는 구조

그래서 보안 관점으로 보면

AS-IS

인증창 호출
↓
결과 수신

(단순)



TO-BE

Access Token
↓
Crypto Token
↓
인증창
↓
StoS 결과조회



그래서 SCI가 요구한 공문에 나온 핵심 이유가 바로 이것입니다.

CI/DI 같은 민감한 본인확인 결과를



브라우저를 통과하지 않고

서버 ↔ 서버 구간에서만 처리하도록 변경

입니다.

주인님이 전에 말했던 "**CI 탈취 위험**"도 사실상 이 StoS 구조를 도입한 가장 큰 이유라고 보면 됩니다.



access token은 즉 cypto token을 사용하기위한 단계.
cypto token은 매 요청마다 다른 요청들 영향이 없도록 요청할때마다 새로 발급되는 암호화키를 발급받는 단계.
stos 는 최종인증이 되면 원하는 정보를 요청해서 받는단계이지만, 기존은 클라이언트로 받아 보안위협이 있었고, 이를 서버대 서버로 넘겨받아 보안을 강화 그래서 Server to Server라서 StoS 이다>??

네 주인님. 거의 95% 맞게 이해하셨습니다.

제가 약간만 다듬어보겠습니다.

1. Access Token

주인님 표현

cypto token을 사용하기 위한 단계



✓ 맞음

조금 더 정확히 말하면

Access Token



=

SCI API를 호출할 수 있는 이용권

입니다.

예를 들면

CLIENT_ID



CLIENT_SECRET

만 가지고는

Crypto Token API



를 직접 못 부릅니다.

먼저

CLIENT_ID



CLIENT_SECRET

↓

Access Token API

↓

access_token

을 받아야

Authorization: Bearer access_token



으로 다음 API를 호출할 수 있습니다.

2. Crypto Token

주인님 표현

매 요청마다 다른 요청들 영향이 없도록



요청할때마다 새로 발급되는 암호화키를 발급받는 단계

✓ 거의 완벽

조금 더 정확히는

암호화키 자체를 발급



하는 건 아니고

암호화키를 생성하기 위한 재료



를 발급합니다.

실제 코드도 보면

```
crypto_token
↓
createSymmetricKey()
↓
```

</> Java



이잖아요.

즉

```
crypto_token
=
대칭키 생성 재료

key, iv
=
실제 암호화 키
```



입니다.

그래서 목적은

거래 단위 격리
(Transaction Isolation)



입니다.

예를 들면

```
A 사용자 인증
crypto_token = AAA
```



B 사용자 인증
crypto_token = BBB

C 사용자 인증
crypto_token = CCC

전부 다름

그래서

=

  ...

↓

B 거래 영향 없음
C 거래 영향 없음



3. StoS

주인님 표현

최종인증이 되면 원하는 정보를 요청해서 받는단계



✓ 맞음

그리고

기존은 클라이언트로 받아 보안위협이 있었고
이를 서버대서버로 넘겨받아 보안을 강화



✓ 이것도 맞음

그리고

Server to Server라서 StoS



✓ 이것도 맞음

실제로

StoS
=
Server to Server



의 약자입니다.

그런데 StoS의 진짜 목적은?

많은 사람들이

결과조회 API



라고만 생각하는데

사실은

민감정보를 브라우저에 노출하지 않는 것



이 핵심입니다.

AS-IS

SCI
↓
브라우저
↓
우리서버



TO-BE

SCI
↓
브라우저
↓
crypto_token_id만 전달
↓
우리서버
↓
StoS API
↓
SCI





CI/DI 수신

차이

AS-IS

CI

DI

이름

생년월일

브라우저 통과



TO-BE

브라우저 통과 안함

서버 ↔ 서버 이동



주인님식으로 한 줄 정리

Access Token

=

SCI API를 사용하기 위한 이용권 발급

Crypto Token

=

이번 인증건만 사용할 암호화 재료 발급
(거래별 분리)

StoS

=

최종 인증결과를 브라우저가 아닌
서버 ↔ 서버 방식으로 조회하여
CI/DI 등의 민감정보 노출을 차단
(Server to Server)



이렇게 이해하시면 이번 구조를 거의 정확하게 이해하신 겁니다.

그리고 SCI가 공문까지 보내면서 기존 방식을 폐기하고 이 방식으로 바꾸라고 한 이유도 결국은:

CI / DI / 본인확인 결과가



브라우저를 통과하는 구조를 없애기 위해서

입니다. 이게 가장 큰 목적입니다.

     ...  출처